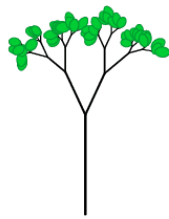


Goldsmiths, University of London

MSc CGP — Maths & Gfx1

Coursework 1: L-Systems

# A Simple L-System Model



Ruyan Qin

Programme: MSC Virtual and Augmented Reality Date: 15th Nov 2025

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>2</b>  |
| <b>2</b> | <b>Features implemented</b>                           | <b>2</b>  |
| 2.1      | Configurable Tree . . . . .                           | 2         |
| 2.2      | Interactive Parameter Adjustment . . . . .            | 3         |
| 2.3      | Stochastic L-system Support (bonus feature) . . . . . | 3         |
| 2.4      | Leaf and Fruit Rendering (Bonus feature) . . . . .    | 4         |
| 2.5      | Realistic Branch Tapering (Bonus feature) . . . . .   | 4         |
| <b>3</b> | <b>Implementation</b>                                 | <b>4</b>  |
| 3.1      | System architecture . . . . .                         | 4         |
| 3.2      | Data Class . . . . .                                  | 5         |
| <b>4</b> | <b>Result</b>                                         | <b>6</b>  |
| <b>5</b> | <b>Discussion</b>                                     | <b>10</b> |
| <b>6</b> | <b>Future Work</b>                                    | <b>10</b> |
| <b>7</b> | <b>Conclusion</b>                                     | <b>10</b> |

## Abstract

This project presents a configurable L-system tree generator built in Python with the Turtle Graphics library. It supports parameter configuration and real-time interaction. Six classic 2D plant models were recreated, and two new variations were added, showing the system's flexibility and creative potential.

## 1 Introduction

This project implements a configurable L-system tree generator using Python and the Turtle Graphic library, the project recreated six simple Deterministic Context-Free 2D (plant) L-systems following the assignments requirements and extends two new configurable trees. This system also integrates interactive controls, allowing users to adjust parameters such as branching angle, iteration depth, and branch length in real time. Furthermore, the system includes stochastic and parameterized extensions, supporting random rule selection and variable branch lengths to simulate the natural variability of plant growth.

The main objectives of this project are:

- Implement L-System generator capable of generating various plants structure based on external configuration,
- Support real-time user interaction through keyboard controls, allowing dynamic adjustment of tree parameters.
- Recreate the six examples of simple Deterministic Context-Free 2D (plant) L-systems as required by the coursework.
- Extend the basic deterministic model by adding stochastic and parameterized behaviors, improving realism and diversity in growth patterns.
- Visualize the system using Turtle Graphics in an interactive way.

## 2 Features implemented

### 2.1 Configurable Tree

The system allows loading external configuration files (tree1.txt-tree9.txt) that defines axiom, production rules, angle, step size, and iteration depth. The symbols are defined as follows:

- **F** – Draws forward, representing branch growth.
- **X** – A recursive non-drawing symbol used for rule expansion.
- **+** – Rotates left by a specified angle.
- **-** – Rotates right by a specified angle.
- **[** – Pushes the current position and heading onto the stack (start a branch).

- **]** – Pops the position and heading from the stack (end a branch).
- **L** – Draws a leaf (custom extension).
- **R** – Draws a fruit (custom extension).
- **random\_length** – A Boolean parameter that controls whether branch lengths are randomized.

Among all 10 configurations, **tree a–tree f** recreate various classical 2D plant L-System structures [1]. **Tree g** introduces randomized fruit generation as an extension of **tree e**, demonstrating stochastic color and size variations. **Tree h** is a stochastic adaptation of the classic branching system from Prusinkiewicz and Lindenmayer [1], originally defined deterministically. In this modified version, the rule

$$X \rightarrow F[+X][-XL] : 0.5 \mid F[+XL][-X] : 0.5$$

introduces probabilistic branching, where the system randomly decides whether leaves (**L**) grow on the left or right branch. The addition of symbol **L** for leaves and the stochastic mechanism together enhance the realism of plant growth simulation. For tree I and tree J, the rules are refer to aul Bourke’s L-System Weed rule and Bushes rule [2].

## 2.2 Interactive Parameter Adjustment

Users can interact with the system in real time using keyboard controls:

- **Up / Down Arrow:** Adjust the branch rotation angle.
- **Right / Left Arrow:** Increase or decrease the iteration depth.
- **W / S:** Modify the step length (branch length).
- **R:** Toggle the random branch length mode.

These interactive features allow users to dynamically interact with the L-system.

## 2.3 Stochastic L-system Support (bonus feature)

The stochastic mechanism can model the natural variability and structural complexity of plants more accurately [3]. In this project, stochastic rules are implemented by assigning probabilities to multiple productions for the same symbol. The selection can be uniform or based on predefined weighted probabilities. For example, in **tree h**, fruits are generated randomly in different colors, while in **tree h**, both branch length and structure are randomized by default, and leaves grow at the end of branches, shown as Figure 3h and Figure 3i.

## 2.4 Leaf and Fruit Rendering (Bonus feature)

The system extends traditional L-System drawing by integrating leaves (L) and fruits (R). Leaves are drawn as green ellipses, rendered only when the tree depth exceeds a threshold, simulating real plant growth. Fruits are drawn with randomized colors, sizes, and probabilities, representing the natural irregularity of fruit distribution.

## 2.5 Realistic Branch Tapering (Bonus feature)

All generated trees branches become thinner with branch depth, creating a more natural and organic appearance similar to real plant growth.

# 3 Implementation

Following the approach proposed by Rimmington and Alagic[4], who combined L-systems with Turtle Graphics to visualize plant morphology, this project implements a similar visualization pipeline using Python and the Turtle Graphics library for real-time rendering of leaves and fruits.

## 3.1 System architecture

This project consists of four modules: the configuration files, the L-system processor, the graphic render system, and the interaction controller. Figure 1 illustrates these modules and their functions within the system. The configuration files define eight different tree structures. The

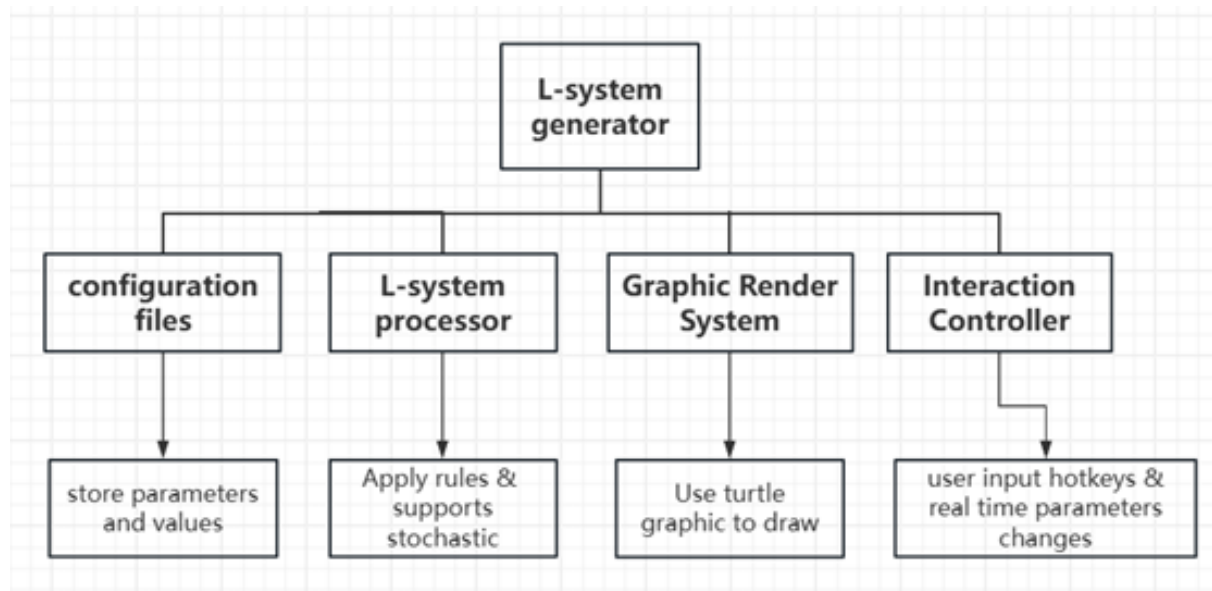


Figure 1: Overview of the L-system generator architecture.

main.py script as the L-system processor, responsible for reading and parsing the configuration files, and then calling turtle graphic engine to visualize and display the trees.

The interaction controller binds keyboards hotkeys that allow user to control parameters in real time, user can adjust the rotation angles, iteration depth and step size, enable or disable random branch length mode.

### 3.2 Data Class



In this system, each major module is defined using the `@dataclass` decorator, which simplifies initialization and improves code readability and modularity.

The `LSystemConfig` class encapsulates all configurable parameters, including the axiom, production rules, rotation angle, iteration count, and drawing step size. It is responsible for loading configuration files and parsing each line to extract parameters, which are then stored in instance variables (see Figure 2a). The `from_file()` class method acts as a factory (shown in Figure 2b), returning a new configuration object containing all parsed values.

The `LSystemGenerator` class handles the growth and iteration logic of the system. The `_pick_production()` method parses each rule string and returns a corresponding replacement. When multiple productions exist, the method processes stochastic rules, selecting one either randomly or based on weighted probabilities.

The `generate()` method handles the iterative rewriting process. In each iteration, it applies the corresponding production rule to every symbol in the current string when the symbol is defined in the rule set (see Figure 2c), progressively expanding the system's structure to simulate the growth of branches.

The `TurtleRenderer` class is responsible for visually rendering the tree structure using the Turtle Graphics library. Each symbol in the L-System string corresponds to a specific drawing action (e.g., F moves forward, + rotates left, - rotates right). The render also defines pen color, stroke width, and line thickness dynamically. It implements branching using a stack mechanism: the turtle's position and heading are pushed onto the stack when encountering [, and popped when

encountering ], returning the turtle to a previous branch point. Symbol L represent leaves, which are drawn only when the depth. Each leaf is rendered as an ellipse, composed of two semicircles with different radius and filled with green color. The symbol R represents fruits, which drawn with randomized probability (`random.random()`), color and size, simulating the natural randomness of fruit distribution.

The `LSystemApp` class acts as the controller and manager between all modules. It initializes the system configuration, sets up the drawing environment, and manages user interactions. It also defined the interactive hotkeys for parameter adjustment, for example, pressing the up arrow increases the branching angle, right arrow increase iteration depth, R key toggles random branch length mode. This interactive controls enables user to dynamically explore different fractal structures.

## 4 Result

In this system , eight configuration files were created, which include different rules and parameters.

To evaluate whether the L-system was implemented and rendered correctly, I referred to the sample plant models presented in *The Algorithmic Beauty of Plants* [1] for comparison. Figure 4 shows the reference examples, while Figure 3(a)–(f) presents the corresponding trees generated by my system. This visual comparison confirms that the generated structures accurately reproduce the tree patterns of the reference samples, demonstrating that the system functions correctly.

For tree g, I modified the configuration of tree e to introduce stochastic features. It should randomly generated colors, random fruit sizes, and a probabilistic decision on whether a fruit should appear at each node. As shown in Figure 3g, the rendered output meets these expectations, confirming that the randomization parameters function as intended.

For tree h, I modified the rule based on inflorescence model. The model should randomly generate green elliptical leaves on either side of the branches and vary the branch lengths for each generation. Figures 3h and 3i show two different renderings of this configuration, It can be observed that the branch lengths and leaf distributions differ between the two results, verifying that the stochastic and structural variation mechanisms work correctly within the system.

For Tree I and Tree J (shown in Figure 3j and Figure 3k), I referred to Paul Bourke’s L-System Weed rule and Bushes rule [2] to recreate the tree pattern. By adjusting the iteration number and step length, the generated structure more closely resembles the original pattern. Compared with the original L-System Weed example by Paul Bourke [2], as shown in Figure 5 and Figure 6, the result appears very similar, indicating that my system successfully reproduced the original pattern.



(a) Tree 1



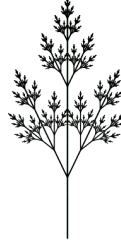
(b) Tree 2



(c) Tree 2



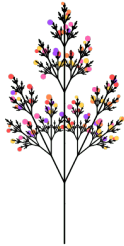
(d) Tree 2



(e) Tree 2



(f) Tree 2



(g) Tree g



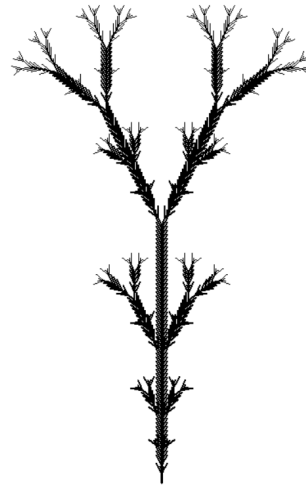
(h) Tree h1



(i) Tree h2



(j) Tree I



(k) Tree J

Figure 3: Examples of generated L-System trees.





**a**  
 $n=5, \delta=25.7^\circ$   
 $F$   
 $F \rightarrow F[+F]F[-F]F$



**b**  
 $n=5, \delta=20^\circ$   
 $F$   
 $F \rightarrow F[+F]F[-F][F]$



**c**  
 $n=4, \delta=22.5^\circ$   
 $F$   
 $F \rightarrow FF-[-F+F+F]+$   
 $[+F-F-F]$



**d**  
 $n=7, \delta=20^\circ$   
 $X$   
 $X \rightarrow F[+X]F[-X]+X$   
 $F \rightarrow FF$



**e**  
 $n=7, \delta=25.7^\circ$   
 $X$   
 $X \rightarrow F[+X][-X]FX$   
 $F \rightarrow FF$



**f**  
 $n=5, \delta=22.5^\circ$   
 $X$   
 $X \rightarrow F-[[X]+X]+F[+FX]-X$   
 $F \rightarrow FF$

Figure 4: Sample L-System plant models from *The Algorithmic Beauty of Plants* [1].



Figure 5: Original example of Paul Bourke's L-System "Weed" pattern [2].

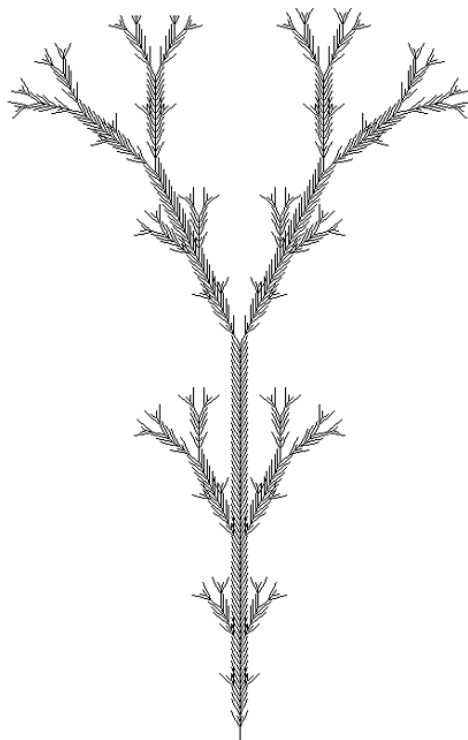


Figure 6: Original example of Paul Bourke's L-System "Bushes" pattern [2].

## 5 Discussion

However, the current system still has limitations. For instance, each time user adjusts the rotation angle via hotkeys, the value changes by a fixed amount and affects all branches simultaneously. Two possible improvements are proposed to address this limitation. The first is to allow users to define and input custom rotation angles manually, which could increase system flexibility. The second is to vary the branching angle at each iteration, following the fractal plants category described by Chandra and Rani[5], in which the scaling factor remains constant while the angle changes. This approach can be implemented by defining a list of rotation angles that change with the iteration depth, so that upper branches spread wider while lower branches grow narrower, producing a more natural appearance.

## 6 Future Work

A further direction for improvement is to develop more complex tree models using parameterized L-systems. Instead of hard-coding fixed rotation angles or step lengths, dynamic parameters could be introduced to allow these values to change continuously during each iteration. As discussed by Rynkiewicz and Napieralski [6], in a parametric L-system every symbol can be followed by one or more parameters enclosed in parentheses (e.g.,  $C(x \cdot e, \frac{e}{2}, a + c, b - d, g)$ ), and mathematical expressions can be used to define growth behavior. This approach would allow gradual variation in branch length and angle over time, producing more realistic structures.

## 7 Conclusion

In conclusion, this project successfully achieved its goal of implementing an interactive L-system generator. Eight configurations were developed in total, six of which accurately reproduce the reference samples from *The Algorithmic Beauty of Plants* [1]. In addition, several extensions were introduced, including stochastic branching, random leaf and fruit generation, and real-time parameter control, which further enhance the system's flexibility and interactivity.

This project also helped me understand how each symbol in the L-system is interpreted and transformed into executable code. It also deepened my understanding of how recursive and stochastic rules work, providing a foundation for designing more complex patterns in the future.

Overall, this project shows that simple procedural rules and interactions can generate fractal, natural-looking tree structures.

## References

- [1] P. Prusinkiewicz and A. Lindenmayer, *The Algorithmic Beauty of Plants* (The Virtual Laboratory), 1st ed. New York, NY: Springer-Verlag New York Inc., 1990, pp. XII, 228, 14 b/w illustrations. Springer Book Archive, ISBN: 978-1-4613-8476-2. DOI: 10.1007/978-1-4613-8476-2.

- [2] P. Bourke, *Blog from paul bourke*, <https://paulbourke.net/fractals/lsys/>, Accessed: 12 Nov. 2025.
- [3] A. Samal, B. Peterson, and D. Holliday, “Recognizing plants using stochastic l-systems,” in *Proceedings of 1st International Conference on Image Processing*, vol. 1, 1994, 183–187 vol.1. DOI: 10.1109/ICIP.1994.413300.
- [4] G. M. Rimmington and M. Alagic, “From modeling foliage with l-systems to digital art,” in *Proceedings of Bridges: Mathematical Connections in Art, Music, and Science*, Available online at: <https://education.wichita.edu/alagic/BTTB/BTTB.htm>, Wichita State University, Kansas, USA, 2007.
- [5] M. Chandra and M. Rani, “Categorization of fractal plants,” *Chaos, Solitons & Fractals*, vol. 41, no. 3, pp. 1442–1447, 2009, ISSN: 0960-0779. DOI: 10.1016/j.chaos.2008.05.024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0960077908002725>.
- [6] F. Rynkiewicz and P. Napieralski, “Procedural fractal plants generation,” *Computer Game Innovations*, p. 24,